



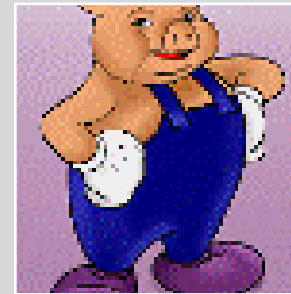
COS20028 – BIG DATA ARCHITECTURE AND APPLICATION

Dr Pei-Wei Tsai (Lecturer, Tutor, and Unit Convenor)
ptsai@swin.edu.au, EN508d





Week 9



Introduction to Apache Pig



INTRODUCTION TO PIG

Introduction to Pig

The key features Pig Offers.

How to use Pig for data processing and analysis.

How to use Pig interactively and in batch mode.

What is Pig?

Apache Pig is a platform for data analysis and processing on Hadoop.

- **It offers an alternative to writing MapReduce code directly.**

Originally developed as a research project at Yahoo!

- **Goal: flexibility, productivity, and maintainability.**
- **Now an open-source Apache project.**

The Anatomy of Pig

- **Main components of Pig**
 - The data flow language (Pig Latin).
 - The interactive shell where you can type Pig Latin statements (Grunt).
 - The Pig interpreter and execution engine.

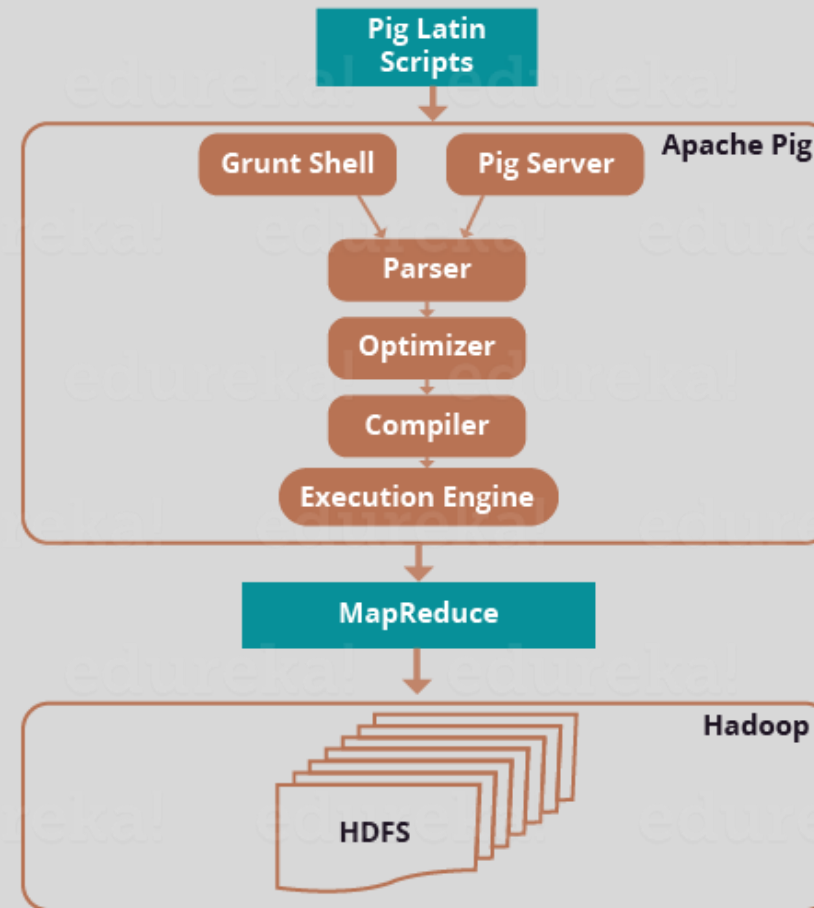


Figure: Apache Pig Architecture

Where to get Pig?

CDH (Cloudera's Distribution including Apache Hadoop) is the easiest way to install Hadoop and Pig.

- **A Hadoop distribution which includes core Hadoop, Pig, Hive, Sqoop, Hbase, Oozie, and other ecosystem components.**
- **Available as RPMs, Ubuntu/Debian/SuSE packages, or a tarball.**
- **Simple installation and 100% free open source.**

Pig's Features

Pig is an alternative to write low-level MapReduce code

Many features enable sophisticated analysis and processing

- **HDFS manipulation.**
- **Unix shell commands.**
- **Relational operations.**
- **Positional references for fields.**
- **Common mathematical functions.**
- **Support for custom functions and data formats.**
- **Complex data structures.**

Use Cases: How are Organisations Using Apache Pig?

Many organisations use Pig for data analysis.

- Finding relevant records in a massive data set.
- Querying multiple data sets.
- Calculating values from input data.

Pig is also frequently used for data processing.

- Reorganising an existing data set.
- Joining data from multiple sources to produce a new data set.

When You Should Use Apache Pig?

If the business use case requires processing multiple data sources, Pig could be an ideal choice.

If the application requires handling time sensitive data then Apache Pig could be a perfect choice.

If the business requires analysis through sampling, Apache Pig should be considered to sample large dataset with a random distribution of data to gain meaningful analytic insights.

Use Cases: Web Log Sessionisation

- Pig can help you extract valuable information from Web server log files.

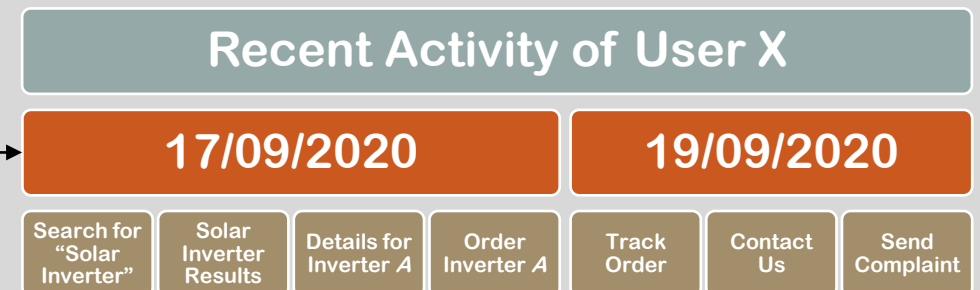
Web Server Log Data

```
10.223.157.186 - - [15/Jul/2009:14:58:59 -0700] "GET / HTTP/1.1" 403 202
10.223.157.186 - - [15/Jul/2009:14:58:59 -0700] "GET /favicon.ico HTTP/1.1" 404
209
10.223.157.186 - - [15/Jul/2009:15:50:35 -0700] "GET / HTTP/1.1" 200 9157
10.223.157.186 - - [15/Jul/2009:15:50:35 -0700] "GET /assets/js/lowpro.js HTTP/1
.1" 200 10469
10.223.157.186 - - [15/Jul/2009:15:50:35 -0700] "GET /assets/css/reset.css HTTP/
1.1" 200 1014
10.223.157.186 - - [15/Jul/2009:15:50:35 -0700] "GET /assets/css/960.css HTTP/1.
1" 200 6206
10.223.157.186 - - [15/Jul/2009:15:50:35 -0700] "GET /assets/css/the-associates.
css HTTP/1.1" 200 15779
10.223.157.186 - - [15/Jul/2009:15:50:35 -0700] "GET /assets/js/the-associates.j
s HTTP/1.1" 200 4492
10.223.157.186 - - [15/Jul/2009:15:50:35 -0700] "GET /assets/js/lightbox.js HTTP
/1.1" 200 25960
10.223.157.186 - - [15/Jul/2009:15:50:36 -0700] "GET /assets/img/search-button.g
if HTTP/1.1" 200 168
10.223.157.186 - - [15/Jul/2009:15:50:36 -0700] "GET /assets/img/dummy/secondary
-news-3.jpg HTTP/1.1" 200 5604
10.223.157.186 - - [15/Jul/2009:15:50:36 -0700] "GET /assets/img/dummy/primary-n
ews-1.jpg HTTP/1.1" 200 10556
```

Process Logs



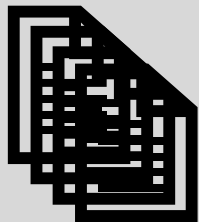
Summary for User Sessions



Use Cases: Data Sampling

- Sampling can help you explore a representative portion of a large data set.
 - Allows you to examine this portion with tools that do not scale with.
 - Supports faster iterations during development of analysis jobs.

Input Data



200 TB

Process Logs



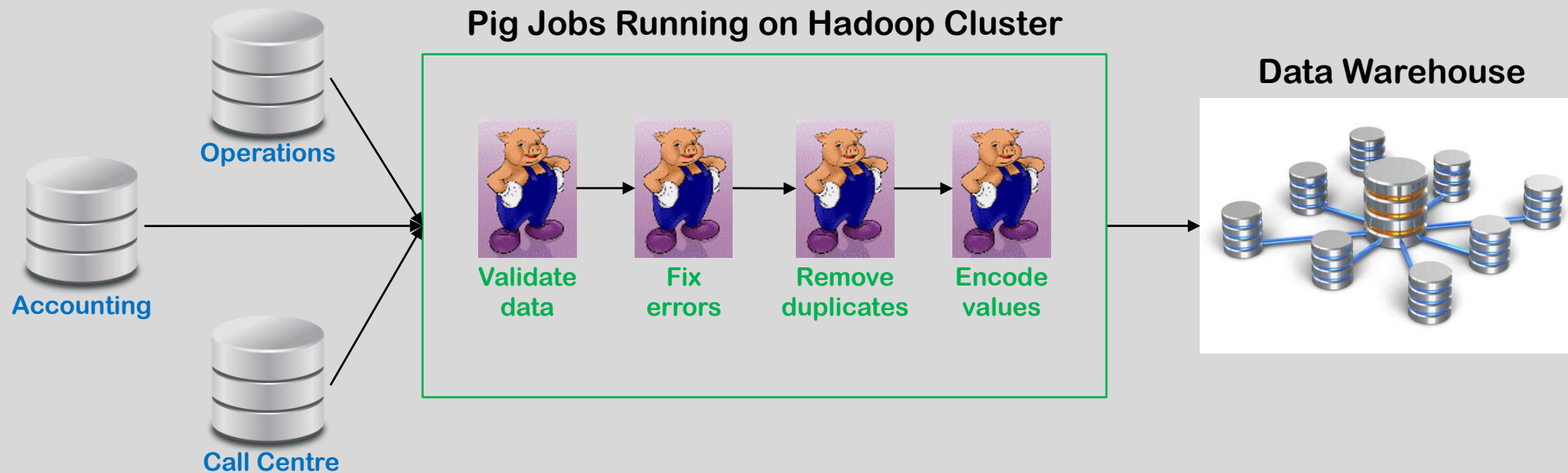
Sampled Data



50 MB

Use Cases: ETL Processing

- Pig is also widely used for Extract, Transform, and Load (ETL) processing



Using Pig Interactively

You can use Pig interactively via the Grunt shell

- Pig interprets each Pig Latin statement as you type it.
- Execution is delayed until output is required.
- Very useful for ad hoc data inspection.

Example of how to start, use, and exit Grunt

```
pig
grunt> allsales = LOAD 'sales' AS (name, price);
grunt> bigsales = FILTER allsales BY price > 100;
grunt> STORE bigsales INTO 'myreport';
grunt> quit;
```

Can also execute a Pig statement from the Unix shell via the -e option.

Interacting with HDFS

- You can interact with HDFS by Pig via the *fs* command.

```
grunt> fs -mkdir sales/;
```

```
grunt> fs -put Europe.txt sales;
```

```
grunt> allsales = LOAD 'sales' AS (name, price);
```

```
grunt> bigsales = FILTER allsales BY price > 100;
```

```
grunt> STORE bigsales INTO 'myreport';
```

```
grunt> fs -getmerge myreport/ bigsales.txt
```


Interacting with Unix

- The *sh* command lets you run Unix programs from Pig:

```
grunt> sh date;
```

```
Thu Oct 09 12:37:46 AEST 2020
```

```
grunt> fs -ls;
```

```
grunt> sh ls;
```

Running Pig Scripts

- A pig script is simply Apache Pig code stored in a text file.
 - Generally, these files have the .pig extension.
- You can run a Pig script from within the Grunt shell via the *run* command.
 - This is useful for automation and batch execution.

```
grunt> run salesreport.pig;
```

- It is common to run a Pig script directly from the Unix shell.

```
$ pig salesreport.pig
```

MapReduce and Local Modes

As described earlier, Apache Pig turns Pig into MapReduce jobs.

- Pig submits those jobs for execution on the Hadoop cluster.

It is also possible to run Pig in “local mode” using the -x flag.

- This runs MapReduce jobs on the *local machine* instead of the cluster.
- Local mode uses the local filesystem instead of HDFS.
- Can be helpful for testing before deploying a job to production.
 - `pig -x local`
 - `pig -x local salesreport.pig`

Client-Side Log Files

If a job fails, Pig produces a log file to explain why.

- **These log files are typically produced in your current working directory on the local (client) machine.**

Essential Points

Pig offers an alternative for writing MapReduce code

- Apache Pig interprets Pig statements in order to create MapReduce jobs.
- It then submits these MapReduce jobs to the Hadoop cluster.

You can execute Pig statements interactively through Grunt

- Pig delays job execution until output is required.

It is also common to store Pig statement in a script for batch execution.

- Allows for automation and code reuse.



BASIC DATA ANALYSIS WITH PIG

Basic Data Analysis with Pig

Basic Syntax of Pig Latin

How to load data using Pig

Which simple data types Pig uses to represent data

How to store and filter data in Pig

How to use many of Pig's built-in functions for data compressing

Apache Pig Overview

Pig Latin is a *data flow* language

- The flow of data is expressed as a sequence of statements.

The following is a simple Pig Latin script to load, filter, and store data.

```
allsales = LOAD 'sales' AS (name, price);
bigsales = FILTER allsales BY price > 1000; -- in UAD

/*
 * Save the filtered results into a new
 * directory, below my home directory.
 */
STORE bigsales INTO 'myreport';
```

Pig Grammar: Keywords

Pig Latin keywords are highlighted here in blue text

- Keywords are reserved – you cannot use them to name things.

```
allsales = LOAD 'sales' AS (name, price);
bigsales = FILTER allsales BY price > 1000; -- in UAD

/*
 * Save the filtered results into a new
 * directory, below my home directory.
 */
STORE bigsales INTO 'myreport';
```

Pig Grammar: Identifiers

Identifiers are the names assigned to fields and other data structures

```
allsales = LOAD 'sales' AS (name, price);
bigsales = FILTER allsales BY price > 1000; -- in UAD

/*
 * Save the filtered results into a new
 * directory, below my home directory.
 */
STORE bigsales INTO 'myreport';
```

Pig Grammar: Identifiers

Identifiers must conform to Pig's naming rules.

An identifier must always begin with a letter.

- This may only be followed by letters, numbers, or underscores.

Valid	x	var1	price_112	MyData
Invalid	2var	price\$	profit%	_sale

Pig Grammar: Comments

Pig supports two types of comments

- Single line comments begin with --
- Multi-line comments begin with /* and end with */

```
allsales = LOAD 'sales' AS (name, price);  
bigsales = FILTER allsales BY price > 1000; -- in UAD
```

```
/*  
 * Save the filtered results into a new  
 * directory, below my home directory.  
 */  
STORE bigsales INTO 'myreport';
```

Case-Sensitive in Pig

Whether case is significant in Pig depends on context.

Keywords (shown here in blue text) are *not* case-sensitive.

- Neither are operators (such as *AND*, *OR*, or *IS NULL*).

Identifiers and paths (shown here in red text) are case-sensitive.

- So are function names (such as *SUM* or *COUNT*) and constants.

```
allsales = LOAD 'sales' AS (name, price);  
bigsales = FILTER allsales BY price > 1000; -- in UAD
```

```
/*  
 * Save the filtered results into a new  
 * directory, below my home directory.  
 */  
STORE bigsales INTO 'myreport';
```

Common Operators in Pig Latin

Many commonly-used operators in Pig are familiar to SQL users.

- Notable difference: Apache Pig uses == and != for comparison.

Arithmetic	Comparison	Null	Boolean
+	==	IS NULL	AND
-	!=	IS NOT NULL	OR
*	<		NOT
/	>		
%	<=		
	>=		

Basic Data Loading in Pig

Pig's default loading function is called *PigStorage*

- The name of the function is implicit when calling *LOAD*.
- *PigStorage* assumes text format with tab-separated columns.

Consider the following file in HDFS called *sales*.

- The two fields are separated by tab characters.

Alice	3122
Ben	3010
Carlos	3810

This example loads data from the above file.

```
allsales = LOAD 'sales' AS (name, price);
```

Data : File and Directories

The previous example loads data from a file named *sales*.

```
allsales = LOAD 'sales' AS (name, price);
```

Since this is not an absolute path, it is relative to your home directory.

- Your home directory in HDFS is typically `/user/your_user_id/`
- Can also specify an absolute path. (e.g., `/dept/sales/2020/q2`)

The path can also refer to a directory.

- In this case, Pig will recursively load all files in that directory.
- File patterns (“globs”) are also supported.

```
allsales = LOAD 'sales_200[6-9]' AS (name, price);
```

Specifying Column Names During Load

The previous example also assigns names to each column

```
allsales = LOAD 'sales' AS (name, price);
```

Assign column names is not required.

- This can be useful when exploring a new dataset.
- Refer to fields by position (*\$0* is first, *\$1* is second, *\$53* is 54th, etc.)

```
allsales = LOAD 'sales';
```


Using Alternate Column Delimiters

You can specify an alternate delimiter as an argument to *PigStorage*.

This example shows how to load comma-delimited data.

- Note that this is a single statement.

```
allsales = LOAD 'sales.csv' USING PigStorage(',') AS (name, price);
```

Or to load pipe-delimited data without specifying column names.

```
allsales = LOAD 'sales.txt' USING PigStorage('|');
```

Simple Data Types in Pig

Pig supports several basic data types.

- Similar to those in most databases and programming languages.

Pig treats fields of unspecified type as an array of bytes.

- Called the *bytearray* type in Pig Latin.

```
allsales = LOAD 'sales' AS (name, price);
```

List of Simple data Types

There are eight data types in Pig for simple values.

Name	Description	Example Value
int	Whole numbers	2020
long	Large whole numbers	5,365,214,142L
float	Decimals	3.14159F
double	Very precise decimals	3.14159265358979323846
boolean	True or false values	true
datetime	Date the time	2020-05-30T14:52:39.000-04:00
chararray	Text strings	Alice
bytearray	Raw bytes (e.g. any data)	N/A

Specifying Data Types in Pig

Pig will do its best to determine data types based on context.

- For example, you can calculate sales commission as *price*0.1*
- In this case, Pig will assume that this value is of type *double*.

However, it is better to specify data types explicitly when possible.

- Helps with error checking and optimisations.
- Easiest to do this upon load using the format *identifier:type*.

```
allsales = LOAD 'sales' AS (name:chararray, price:int);
```

Choosing the right data type is important to avoid loss of precision.

Important: Avoid using floating point numbers to represent money!

HCatalog

You have seen how to specify names, and paths in the *LOAD* statement.

HCatalog is a tool that can store this information permanently.

- So it need not be specified each time.
- Simplifies sharing of metadata between Pig, Hive, Spark SQL, and/or custom MapReduce applications.

You can load data easily after first setting it up with Hcatalog.

```
allsales = LOAD 'sales' USING org.apache.hcatalog.pig.HCatLoader();
```

How Pig Handles Invalid Data?

When encountering invalid data, Pig substitutes *NULL* for the value.

- For example, an *int* field containing the value *Q2*.

The *IS NULL* and *IS NOT NULL* operators test for null values.

- Note that *NULL* is not the same as the empty string''.

You can use these operators to filter out bad records.

```
Hasprices = FILTER Records by price IS NOT NULL;
```

Key Data Concepts in Pig

Relational databases have tables, rows, columns, and fields.

We will use the following data to illustrate Pig's equivalents.

- Assume this data was loaded from a tab-delimited text file as before.

Name	Price	Country
Alice	3128	US
Bill	4214	UK
Carlos	1120	AUS
David	7455	FR
Frank	2594	IT

Pig Data Concepts: Fields and Tuples

A single element of data is called a field.

- It corresponds to one of the eight data types seen earlier.

A collection of values is called a tuple.

- Fields within a tuple are ordered, but need not all be the same type.

Name	Price	Country
Alice	3128	US
Bill	4214	UK
Carlos	1120	AUS
David	7455	FR
Frank	2594	IT

Name	Price	Country
Alice	3128	US
Bill	4214	UK
Carlos	1120	AUS
David	7455	FR
Frank	2594	IT

Pig Data Concepts: Bags

A collection of tuples is called a bag.

Tuples within a bag are unordered by default.

- The field count and types may vary between tuples in a bag.

Name	Price	Country
Alice	3128	US
Bill	4214	UK
Carlos	1120	AUS
David	7455	FR
Frank	2594	IT

Pig Data Concepts: Relations

A relation is simply a bag with an assigned name (alias)

- Most Pig Latin statements create a new relation.

A typical script loads one or more datasets into relations.

- Processing creates new relations instead of modifying existing ones.
- The final result is usually also a relation, stored as the output.

```
allsales = LOAD 'sales' AS (name, price);  
bigsales = FILTER allsales BY price > 1000;  
STORE bigsales INTO 'myreport';
```

Data Output in Pig

The command used to handle output depends on its destination.

- ***DUMP***: sends output to the screen.
- ***STORE***: sends output to disk (HDFS).

Example of ***DUMP*** output, using data from the file shown earlier.

- The parentheses and commas indicate tuples with multiple fields.

```
(Alice,3128,US)
(Bill,4214,UK)
(Carlos,1120,AUS)
(David,7455,FR)
(Frank,2594,IT)
```

Storing Data with Pig

The *STORE* command is used to store data to *HDFS*.

- Similar to *LOAD*, but *writes* data instead of reading it.
- The output path is the name of a directory.
 - The directory must not yet exist.

As with *LOAD*, the use of PigStorage is implicit.

- The field delimiter also has a default value (tab).

```
STORE bigsales INTO 'myreport';
```

- You may also specify an alternate delimiter.

```
STORE bigsales INTO 'myreport' USING PigStorage(',');
```

Viewing the Schema with *DESCRIBE*

The *DESCRIBE* command shows the structure of the data, including names and types.

The following Grunt session shows an example.

```
grunt> allsales = LOAD 'sales' AS (name:chararray,price:int);
```

```
grunt> DESCRIBE allsales;
```

```
allsales: {name: chararray, price: int}
```

Filtering in Pig Latin

The *FILTER* keyword extracts tuples matching the specified criteria.

```
bigsales = FILTER allsales BY price > 3000;
```

allsales				bigsales		
Name	Price	Country		Name	Price	Country
Alice	3128	US	price > 3000 →	Alice	3128	US
Bill	4214	UK		Bill	4214	UK
Carlos	1120	AUS				
David	7455	FR		David	7455	FR
Frank	2594	IT				

Filtering by Multiple Criteria

You can combine criteria with *AND* and *OR*

Somesales = FILTER allsales BY name == 'Carlos' OR (price > 3500 AND price < 8000);

allsales		
Name	Price	Country
Alice	3128	US
Bill	4214	UK
Carlos	1120	AUS
David	7455	FR
Frank	2594	IT

bigsales		
Name	Price	Country
Bill	4214	UK
Carlos	1120	AUS
David	7455	FR

Name is Carlos, or price is greater than 3500 and less than 8000.

Aside: String Comparisons in Pig Latin

The `==` operator is supported for *any* type in Pig Latin.

- This operator is used for exact comparisons.

```
allsales = FILTER allsales BY name == 'Alice';
```

Pig Latin supports pattern matching through Java's *regular expressions*.

- This is done with the ***MATCHES*** operator.

```
a_names = FILTER allsales BY name MATCHES 'A.*';
```

```
Spammers = FILTER senders BY email_addr MATCHES '.*@example\\.com$';
```

Field Selection in Pig Latin

Filtering extracts rows, but sometimes we need to extract columns.

- This is done in Pig using the *FOREACH* and *GENERATE* keywords.

```
twofields = FOREACH allsales GENERATE amount, trans_id;
```

allsales		
employee	amount	trans_id
Alice	3128	2157546
Bill	4214	2157547
Carlos	1120	2157548
David	7455	2157549
Frank	2594	2157549



twofields	
amount	trans_id
3128	2157546
4214	2157547
1120	2157548
7455	2157549
2594	2157549

Generating New Fields in Pig Latin

The *FOREACH* and *GENERATE* keywords can also be used to *create* fields.

- For example, you could create a new field based on price.

```
t = FOREACH allsales GENERATE price * 0.07;
```

It is possible to name such fields.

```
t = FOREACH allsales GENERATE price * 0.07 AS tax;
```

And you can also specify the data type:

```
t = FOREACH allsales GENERATE price * 0.07 AS tax:float;
```

Eliminating Duplicates

DISTINCT eliminates duplicate records in a bag.

- All fields must be equal to be considered a duplicate.

```
unique_records = DISTINCT all_alices;
```

all_alices

firstname	lastname	country
Bill	Jones	US
Bill	Smith	US
Bill	Brown	AU
Bill	Brown	AU
Bill	Brown	CA



unique_records

firstname	lastname	country
Bill	Jones	US
Bill	Smith	US
Bill	Brown	AU
Bill	Brown	CA

Controlling Sort Order

Use *ORDER...BY* to sort the tuples in a bag in ascending order.

- Add *DESC* to sort in descending order instead.
- Be careful to specify a schema because the data type affects how data is sorted!

```
sortedsales = ORDER allsales BY country DESC;
```

allsales		
name	price	country
User1	21.48	US
User2	36.25	CA
User3	27.66	AU
User4	23.67	IT
User5	65.46	FR



sortedsales		
name	price	country
User1	21.48	US
User4	23.67	IT
User5	65.46	FR
User2	36.25	CA
User3	27.66	AU



Limiting Results

As the same as in SQL, you can use *LIMIT* to reduce the number of output records.

```
somesales = LIMIT allsales 10;
```

Record ordering is random unless specified with *ORDER BY*.

- Use *ORDER BY* and *LIMIT* together to find top-N results.

```
sortedsales = ORDER allsales BY price DESC;  
top_five = LIMIT sortedsales 5;
```

Built-in Functions

These are just a sampling of Pig's many built-in functions.

Function Description	Example Invocation	Input	Output
Convert to uppercase	UPPER(country)	uk	UK
Remove leading/trailing spaces	TRIM(name)	_Bob	Bob
Return a random number	RANDOM()		0.42962348
Round to closet whole number	ROUND(price)	37.109	37
Return chars between two positions	SUBSTRING(name, 0, 2)	King	Ki

You can use these with the *FOREACH...GENERATE* keywords.

```
rounded = FOREACH allsales GENERATE ROUND(price);
```




PROCESSING COMPLEX DATA WITH PIG

Data Compression with Pig

Handling Compression

- Apache Pig supports compression via the load and the store functions.
- PigStorage and TextLoader support gzip and bzip compression for both read and write.
- To work with bzip compression files, the input/output files need to have a .bz or .bz2 extension.

```
allsales = LOAD 'sales.gz';  
STORE allsales INTO 'myoutput.gz';
```

Processing Complex Data with Pig

Main Topics:

- How does Pig use bags, tuples, and maps to represent complex data?
- Pig techniques for grouping and ungrouping data.
- Using aggregate functions in Pig Latin.
- Iterating through records in complex data structures.

Storage Formats

We have seen that *PigStorage* loads and stores data.

- Uses a delimited text file format.

```
allsales = LOAD 'sales' AS (name, price);
```

- The default delimiter (tab) can be easily changed.

```
allsales = LOAD 'sales' USING PigStorage(',') AS (name, price);
```

Sometimes you need to load or store data in other formats.

Other Supported Formats

Here are a few of Pig's built-in functions for loading data in other formats.

- It is also possible to implement a custom loader by writing Java code.

Name	Loads Data From
TextLoader	Text files (entire line as one field)
JsonLoader	Text files containing JSON-formatted data
BinStorage	File containing binary data
HBaseLoader	Hbase, a scalable NoSQL database built on Hadoop

Load and Store Functions

Some functions load data, some store data, and some do both.

Load Function	Equivalent Store Function
PigStorage	PigStorage
TextLoader	<i>None</i>
JsonLoader	JsonStorage
BinStorage	BinStorage
HBaseLoader	HBaseStorage

Pig's Complex Data Types: Tuple and Bag

We have already seen two of Pig's three complex data types:

- A tuple is a collection of values.
- A bag is a collection of tuples.

trans_id	total	salesperson
1055746	2979	Alice
1055747	365	Bill
1055748	2364	Carlos
1055749	1141	Peter
1055750	2264	Gary
1055751	3541	Mike

← tuple

← bag

Pig's Complex Data Type: Map

Pig also supports another complex type: Map

- A map associated a *chararray* (key) to another data element (value).

trans_id	amount	salesperson	sales_details
105845	2367	Alice	Date → 12-03-2020 TID → 34020 Store → HWT02
105846	3342	Bill	Date → 12-03-2020 TID → 34020 Store → HWT02 Coupon → cKx2uTz
105847	2768	Charlie	Date → 12-03-2020 TID → 34020 Store → HWT02

Representing Complex Types in Pig

It is important to know how to define and recognise these types in Pig.

Type	Definition
Tuple	Comma-delimited list inside parentheses: (<code>'1055746'</code> , 2979, <code>'Alice'</code>)
Bag	Braces surround comma-delimited list of tuples: {(<code>'1055746'</code> , 2979, <code>'Alice'</code>), (<code>'1055747'</code> , 365, <code>'Bill'</code>)}
Map	Brackets surround comma-delimited list of pairs; Keys and values separated by #: [<code>'store' #'HWT01'</code> , <code>'location' #'Hawthorn'</code>]

Loading and Using Complex Types

Complex data types can be used in any Pig field.

The following example show how a bag is stored in a text file.

- Example: Transaction ID, amount, items sold (a bag of tuples)

1055746	2979	{('74232', 2001), ('37301', 236)}
---------	------	-----------------------------------

Field 1	tab	Field 2	tab	Field 3
---------	-----	---------	-----	---------

Here is the corresponding LOAD statement specifying the schema.

```
details = LOAD 'salesdetail' AS (trans_id:chararray, amount:int, items_sold:bag {item:tuple (TID:chararray, price:int)});
```

Loading and Using Complex Types

The following example show how a map is stored in a text file.

- Example: Customer name, credit account details (map), year account opened.

Ted	[creditlimit#5800,creditused#200]	2020
Field 1	Field 2	Field 3

Here is the corresponding *LOAD* statement specifying the schema.

```
credit = LOAD 'customer_accounts' AS (name:chararray, account:map[], year:int);
```

Referencing Map Data

Consider a file with the following data

```
Ted      [salary#5000,age#43]
```

And loaded with the following schema

```
details = LOAD 'data' AS (name:chararray, info:map[]);
```

Here is the syntax for referencing data within the map and bag

```
salaries = FOREACH details GENERATE info#'salary';
```

Grouping Records by a Field

Sometimes you need to group records by a given field

- For example, you can calculate commissions for each employee.

Alice	729
Bill	3128
Alice	8243
Carlos	22403
Carlos	4028

Using *GROUP BY* to do this in Pig Latin

```
grunt> byname = GROUP sales BY name;
```

Grouping Records by a Field

The new relation always contains two fields

```
grunt> byname = GROUP sales BY name;  
grunt> DESCRIBE byname;  
Byname: {group: chararray,sales: {(name:chararray,price:int)}}
```

The first field is *literally* named *group* in all cases.

- Contains the value from the field specified in *GROUP BY*

The second field is named after the relation specified in *GROUP BY*

- It's a bag containing one tuple for each corresponding value.

Grouping Records by a Field

The example below shows the data after grouping

Alice	2874
Bob	3929
Alice	2354
Carlos	28241
Carlos	44265

Input Data (sales)

```
grunt> byname = GROUP sales BY name;  
grunt> DUMP byname;  
(Bob, {(Bob,3929)})  
(Alice, {(Alice,2874), (Alice,2354)})  
(Carlos, {(Carlos,28241), (Carlos,44265)})
```

group
field

sales field

Using *GROUP BY* to Aggregate Data

Aggregate functions create one output value from multiple input values

- For example, to calculate total sales by employee.
- Usually applied to grouped data.

```
grunt> byname = GROUP sales BY name;  
grunt> DUMP byname;  
(Bob, {(Bob,3929)})  
(Alice, {(Alice,2874), (Alice,2354)})  
(Carlos, {(Carlos,28241), (Carlos,44265)})
```

```
grunt> totals = FOREACH byname GENERATE group, SUM(sales.price);  
grunt> dump totals;  
(Bob,3929)  
(Alice,5228)  
(Carlos,72506)
```

Grouping Everything Into a Single Record

We just saw that *GROUP BY* creates one record for each unique value.

GROUP ALL puts *all* data into one record.

```
grunt> grouped = GROUP sales ALL;  
grunt> DUMP grouped;  
(all, {(Alice,2874), (Bob,3929), (Alice,2354), (Carlos,28241), (Carlos,44265)})
```


Using *GROUP ALL* to Aggregate Data

Use *GROUP ALL* when you need to aggregate one or more columns.

- For example, to calculate total sales for all employees:

```
grunt> grouped = GROUP sales ALL;  
grunt> DUMP grouped;  
(all, {(Alice,2874), (Bob,3929), (Alice,2354), (Carlos,28241), (Carlos,44265)})  
  
grunt> totals = FOREACH grouped GENERATE SUM(sales.price);  
grunt> dump totals;  
(81663)
```

Removing Nesting in Data

Some operations in Pig, like grouping produce nested data structures.

```
grunt> byname = GROUP sales BY name;  
grunt> DUMP byname;  
(Bob, {(Bob,3929)})  
(Alice, {(Alice,2874), (Alice,2354)})  
(Carlos, {(Carlos,28241), (Carlos,44265)})
```

Grouping can be useful to supply data to aggregate functions.

However, sometimes you want to work with a “flat” data structure.

- The *FLATTEN* operator removes a level of nesting in data.

An Example of *FLATTEN*

The following shows the nested data and what *FLATTEN* does to it

```
grunt> byname = GROUP sales BY name;  
grunt> DUMP byname;  
(Bob, {(Bob,3929)})  
(Alice, {(Alice,2874), (Alice,2354)})  
(Carlos, {(Carlos,28241), (Carlos,44265)})
```

```
grunt> flat = FOREACH byname GENERATE group, FLATTEN(sales.price);  
grunt> DUMP flat;  
(Bob,3929)  
(Alice,2874)  
(Alice,2354)  
(Carlos,28241)  
(Carlos,44265)
```

Pig's Built-In Aggregate Functions

- Pig has built-in support for other aggregate functions besides SUM.
- Example:
 - *AVG*: Calculates the average (mean) of all values.
 - *MIN*: Returns the smallest value.
 - *MAX*: Returns the largest value.
- Pig has two built-in functions for counting records.
 - *COUNT*: Returns the number of *non-null* elements in the bag.
 - *COUNT_STAR*: Returns the number of *all* elements in the bag.

Other Notable Built-in Functions

- Here are a some other useful Pig functions:
 - See the Pig documentation for a complete list.

Function	Description
DIFF	Finds tuples that appear in only one of two supplied bags.
IsEmpty	Used with <i>FILTER</i> to match bags or maps that contain no data.
SIZE	Returns the size of the field (definition of size varies by data type)
TOKENIZE	Splits a text string (chararray) into a bag of individual words.

Record Iteration

We have seen that *FOREACH...GENERATE* iterates through records.

The goal is to transform records to produce a new relation.

- Sometimes to select only certain columns.

```
price_column_only = FOREACH sales GENERATE price;
```

- Sometimes to create new columns.

```
taxes = FOREACH sales GENERATE price * 0.07;
```

- Sometimes to invoke a function on the data.

```
Totals = FOREACH grouped GENERATE SUM(sales.price);
```

Nesting the FOREACH Keyword

A variation on FOREACH applies a set of operations to each record.

- This is often used to apply a series of transformations in a group.

This is called a “nested FOREACH”

- Allows only relational operations (e.g. *LIMIT, FILTER, ORDER BY*)
- *GENERATE* must be the last line in the block.

Nested *FOREACH* Example

Input Data

CEO	227,552
Director	182,515
Director	182,515
Director	184,515
Manager	165,117
Manager	165,282
Manager	145,379
Engineer	75,000
Engineer	75,000
Engineer	76,000

Our input data contains a list of employee job titles and corresponding salaries.

Goal: identify the top three salaries within each title.

Nested *FOREACH* Example

First load the data from the file.

Next, group employees by title.

- Assigned to new relation `title_group`

```
employees = LOAD 'data' AS (title:chararray, salary:int);  
title_group = GROUP employees BY title;
```

```
top_salaries = FOREACH title_group {sorted = ORDER employees  
BY salary DESC; highest_paid = LIMIT sorted 3; GENERATE  
group, highest_paid; };
```

Input Data

CEO	227,552
Director	182,515
Director	182,515
Director	184,515
Manager	165,117
Manager	165,282
Manager	145,379
Engineer	75,000
Engineer	75,000
Engineer	76,000

Nested *FOREACH* Example

Then nested *FOREACH* iterates through every record in the group (i.e., each job title).

- It sorts each record in that group in descending order of salary.
- It then selects the top three.
- *GENERATE* outputs the title and salaries.

```
employees = LOAD 'data' AS (title:chararray, salary:int);  
title_group = GROUP employees BY title;
```

```
top_salaries = FOREACH title_group {sorted = ORDER employees  
BY salary DESC; highest_paid = LIMIT sorted 3; GENERATE  
group, highest_paid; };
```

Input Data

CEO	227,552
Director	182,515
Director	182,515
Director	184,515
Manager	165,117
Manager	165,282
Manager	145,379
Engineer	75,000
Engineer	75,000
Engineer	76,000

Nested *FOREACH* Example

- Code (*LOAD* statement removed for brevity)

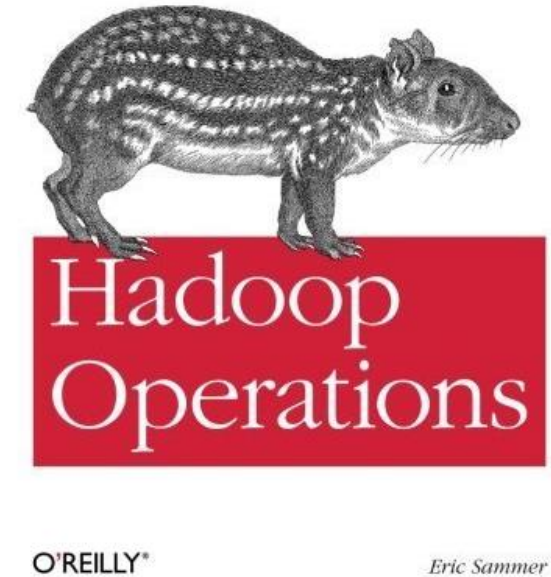
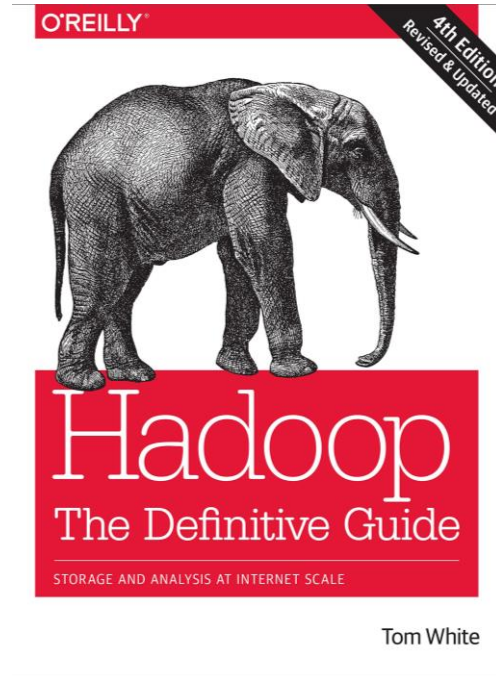
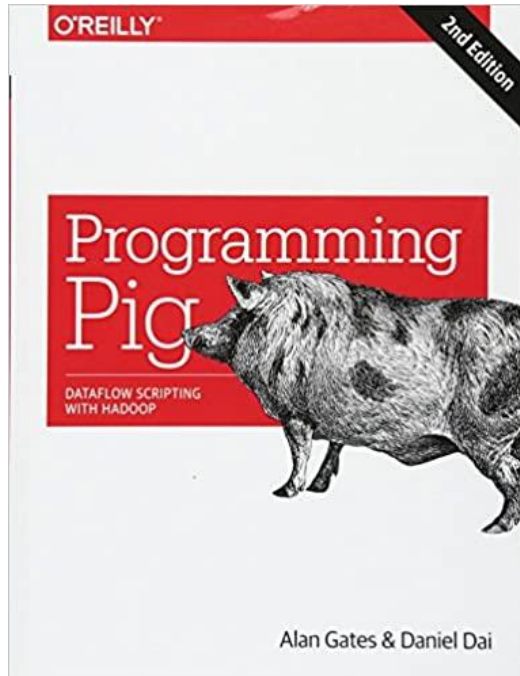
```
title_group = GROUP employees BY title;  
  
top_salaries = FOREACH title_group {  
    sorted = ORDER employees BY salary DESC;  
    highest_paid = LIMIT sorted 3;  
    GENERATE group, highest_paid;  
};
```

Input Data

CEO	227,552
Director	182,515
Director	182,515
Director	184,515
Manager	165,117
Manager	165,282
Manager	145,379
Engineer	75,000
Engineer	75,000
Engineer	76,000

- Output produced by *DUMP top_salaries*

```
(CEO, {(CEO,227552)})  
(Director, {(Director,184515), (Director, 182515), (Director, 182515)})  
(Manager, {(Manager,165117), (Manager,165282), (Manager,145379)})  
(Engineer, {(Engineer,76000), (Engineer,75000), (Engineer,75000)})
```



Texts and Resources

- Unless stated otherwise, the materials presented in this lecture are taken from:
 - Hadoop: the definitive guide, White, Tom (Tom E.) author., 4th ed., Sebastopol, California: O'Reilly, 2015.
 - Hadoop operations, Sammer, Eric., Loukides, Michael Kosta.; Nash, Courtney.; Romano, Robert (Illustrator), illustrator., First edition., Sebastopol, CA: O'Reilly, 2012.
 - Programming pig: dataflow scripting with hadoop, Gates, Alan, author.; Dai, Daniel, 2nd edition., Beijing, China: O'Reilly, 2017



Unit Summary

- MapReduce Code Explanation.

Summary

